2600
2600, école de cybersécurité

MODULE
Noyau Windows

TP 01

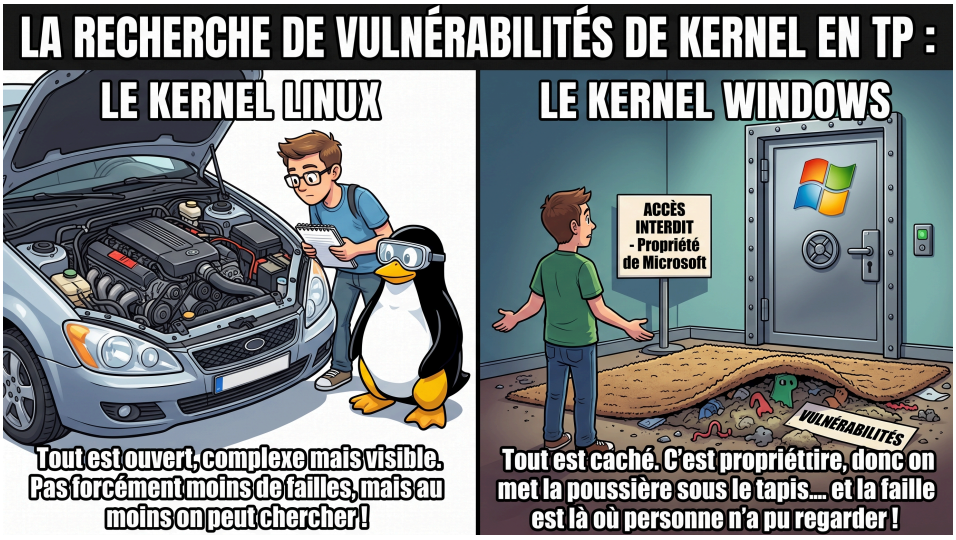
Setup, WinDBG, BYOLD & Fuzzing

Environnement de développement, débogage kernel et analyse de vulnérabilités drivers

AUTEUR
Lionel Auroux

DATE
2026

DURÉE
5h



Sommaire

Objectifs 4
Prérequis 4
Rendu attendu 5
Partie 1 — Mise en place (1h) 5
1.1 : Préparer la VM Windows 5
1.2 : Installer les outils 5
1.3 : Connexion WinDBG 6
1.4 : Premier driver — Hello Kernel 6
1.5 : Charger le driver 7
Partie 2 — WinDBG : commandes essentielles (1h) 7
2.1 : Lister les modules 7
2.2 : Explorer les symboles 8
2.3 : Afficher les structures 8
2.4 : Lister les processus et explorer la liste chaînée 8
2.5 : Inspecter votre driver et les tokens 9
Partie 3 — BYOLD : Utilisation offensive de procexp.sys (1h30) 9
Contexte 9
Partie A — Extraction et analyse du driver (30min) 10
Étape 1 : Obtenir le driver 10
Étape 2 : Analyser le binaire PE 10
Étape 3 : Analyser dans WinDBG 11
Partie B — Écrire un « process killer » avec procexp.sys (45min) 11
Objectif 11
Pourquoi le driver est nécessaire 11
Chaîne d'attaque 11
IOCTLs documentés 12
Ce que vous devez implémenter 12
Comportement selon la version 13
Vérification 13
Partie C — Fuzzing de procexp.sys (15min) 14
Partie D — Analyse de la menace et des défenses (15min) 14
Partie 4 — Fuzzing d'IOCTL en C userland (1h30) 14
Contexte 14
Objectifs 15
Rappel : structure d'un CTL_CODE 15
Ce que vous devez implémenter 15
Vérification 16
Test sur votre Hello Driver 16
Test sur procexp.sys 16
Observation dans WinDBG 17
Pour aller plus loin (bonus) 17
Fuzzer intelligent 17
Barème indicatif 17
Annexe — Compilation sans Visual Studio 18

Troubleshooting 18

Objectifs

L'objectif de cet atelier est de mettre en place un environnement de développement kernel Windows, de découvrir WinDBG, et d'analyser les vulnérabilités des drivers via le BYOLD et le fuzzing :

- Configurer une VM Windows en mode testsigning et debug
- Compiler et charger un premier driver
- Maîtriser les commandes essentielles de WinDBG
- Analyser un scénario BYOLD (Bring Your Own Legitimate Driver)
- Écrire un fuzzer d'IOCTL en C userland



Fig. 1. – width:400px

Prérequis

- Une VM Windows 10/11 installée (Hyper-V, VMware ou VirtualBox)
- Un poste hôte avec accès réseau à la VM
- Visual Studio Community ou supérieur installé sur la VM (ou sur l'hôte si compilation croisée)
- Télécharger : Windows Driver Kit (WDK), WinDBG Preview, Sysinternals Suite
- Maîtrise de la programmation C (compilation, pointeurs, structures) — le fuzzing se fait en C userland

Rendu attendu

À la fin de ce TP, vous devez rendre une archive `tp01_NOM_PRENOM.zip` contenant :

Fichier	Description
`rapport_tp01.md`	Réponses aux questions, captures d'écran annotées, observations
`img/*`	chemin des captures d'écran
`README.md`	Documentation sur la compilation de hello.sys et ioctl_fuzzer.exe
`hello/hello.c`	Code source du driver Hello Kernel
`procexp_kill/procexp_kill.c`	Programme de kill via procexp.sys
`fuzzer/ioctl_fuzzer.c`	Code source du fuzzer d'IOCTL

Le rapport doit contenir **les réponses à toutes les questions numérotées** (Q1, Q2, ...) et des **captures d'écran** montrant le fonctionnement.

Partie 1 — Mise en place (1h)

1.1 : Préparer la VM Windows

Installer une VM Windows 10/11 (Hyper-V, VMware ou VirtualBox). Activer le mode testsigning et debug :

```
code

bcdedit /set testsigning on
bcdedit /debug on
bcdedit /dbgsettings net hostip:<IP_HOTE> port:50000 key:1.2.3.4
```

Redémarrer la VM. Un watermark « Test Mode » doit apparaître en bas à droite.

1.2 : Installer les outils

Sur la machine hôte (ou sur la VM selon votre configuration) :

- **vsBuild** (Community ou supérieur) avec le workload « Desktop development with C++ »
- **Microsoft Entreprise Windows Driver Kit (WDK)** correspondant à la version de Windows de la VM
- **WinDBG** (version Preview recommandée : disponible sur le Microsoft Store)
- **Sysinternals Suite** : DbgView, WinObj, Process Explorer

1.3 : Connexion WinDBG

Lancer WinDBG et configurer une connexion kernel debugging :

```
code


File → Attach to kernel → Net
Port: 50000
Key: 1.2.3.4
```

Vérifier que la connexion fonctionne : WinDBG doit afficher `Connected to target` après un `break` (Ctrl+Break ou Debug → Break).

1.4 : Premier driver — Hello Kernel

Créer un projet Visual Studio de type « Kernel Mode Driver, Empty (KMDF) » ou « Empty WDM Driver ».

Écrire un fichier `hello.c` :

```c
#include <ntddk.h>

void HelloUnload(PDRIVER_OBJECT DriverObject)
{
    UNREFERENCED_PARAMETER(DriverObject);
    DbgPrint("Hello: Driver unloaded\n");
}

NTSTATUS DriverEntry(PDRIVER_OBJECT DriverObject,
                     PUNICODE_STRING RegPath)
{
    UNREFERENCED_PARAMETER(RegPath);
    DbgPrint("Hello: Driver loaded!\n");
    DriverObject->DriverUnload = HelloUnload;
    return STATUS_SUCCESS;
}
```

Charger l'iso de Entreprise WDK:

- monter l'ISO sur Virtual Box
- Lancer un cmd.exe et aller dans le drive D: du CD
- Lancer LaunchBuildEnv.cmd

1) Écriture du code en C
2) pour fabriquer son code. `msbuild MyDriver.vcxproj /p:Configuration=Debug /p:Platform=x64`
3) Besoin de définir un fichier *.vcxproj pour WDM. Utiliser `template_driver_win64.vcxproj`.

Compiler le driver en configuration **x64 Debug**. Copier le `.sys` dans la VM via un dossier partagé, un transfert réseau, ou un chemin UNC.

Q1 : Quel est le *subsystem* d'un fichier `.sys` ? Vérifiez avec `dumpbin /headers hello.sys` ou un outil PE (PE-bear, CFF Explorer). Notez la valeur dans votre rapport.

1.5 : Charger le driver

Dans la VM, ouvrir un terminal administrateur :

```
code

sc create HelloDriver type= kernel binPath= C:\chemin\vers\hello.sys
sc start HelloDriver
```

Observer le message dans **DbgView** (Sysinternals) : `Hello: Driver loaded!`

Décharger :

```
code

sc stop HelloDriver
sc delete HelloDriver
```

Q2 : Que se passe-t-il si vous tentez `sc start HelloDriver` sans avoir activé `testsigning` ? Quel est le code d'erreur retourné ? Expliquez pourquoi.

Q3 : Listez les étapes complètes (de la compilation au message DbgView) dans votre rapport, avec une capture d'écran de DbgView montrant le message.

Partie 2 — WinDBG : commandes essentielles (1h)

2.1 : Lister les modules

Dans WinDBG (connecté au kernel de la VM) :

```
code

kd> lm
```

Identifier `nt` (ntoskrnl.exe), `hal`, `fltmgr`, et votre driver `HelloDriver`.

2.2 : Explorer les symboles

```
code

kd> x nt!Ps*
```

Q4 : Combien de fonctions commençant par `Ps` trouvez-vous ? Parmi elles, identifiez et décrivez brièvement le rôle de :

- `PsLookupProcessByProcessId`
- `PsSetCreateProcessNotifyRoutine`
- `PsGetCurrentProcess`

2.3 : Afficher les structures

```
code

kd> dt nt!_EPROCESS
```

Observer les champs : `UniqueProcessId`, `ActiveProcessLinks`, `Token`, `ImageFileName`, `Peb`.

```
code

kd> dt nt!_DRIVER_OBJECT
```

Observer les champs : `MajorFunction`, `DriverName`, `DeviceObject`.

Q5 : Notez les offsets des champs suivants de `_EPROCESS` dans votre rapport. Ces offsets seront réutilisés dans les TP suivants :

- `UniqueProcessId` : offset = ?
- `ActiveProcessLinks` : offset = ?
- `Token` : offset = ?
- `ImageFileName` : offset = ?

2.4 : Lister les processus et explorer la liste chaînée

```
code

kd> !process 0 0
```

Trouver le processus **System** (PID 4). Noter son adresse EPROCESS.

Parcourir manuellement la liste `ActiveProcessLinks` :

```
code

kd> dt nt!_EPROCESS ActiveProcessLinks.Flink <adresse_system>
```

Soustraire l'offset de `ActiveProcessLinks` à cette adresse pour obtenir le EPROCESS suivant, puis afficher son `ImageFileName`. Répéter 3 fois.

Q6 : Que se passerait-il si on modifiait les pointeurs Flink/Blink pour « sauter » un processus de cette liste ? Le processus serait-il encore visible dans le Task Manager ? Continuerait-il de s'exécuter ? Justifiez.

2.5 : Inspecter votre driver et les tokens

Charger votre driver `HelloDriver` puis dans WinDBG :

```
code

kd> !drvobj \Driver\HelloDriver 2
```

Explorer les tokens : comparer le token de **System** (PID 4) et de **cmd.exe** :

```
code

kd> !process 0 0 System
kd> dt nt!_EPROCESS Token <adresse_system>
kd> !token <adresse_token>
```

Q7 : Listez au moins 3 privilèges que System possède et que cmd.exe ne possède pas. Que se passerait-il si on copiait le token de System dans le champ Token de cmd.exe ?

Partie 3 — BYOLD : Utilisation offensive de procexp.sys (1h30)

Contexte

Le **BYOLD** (Bring Your Own Legitimate Driver) consiste à utiliser un driver légitime signé à des fins malveillantes. Contrairement au BYOVD (driver vulnérable), le driver fonctionne **exactement comme prévu** — c'est son usage qui est détourné.

Le driver `PROCEXP152.sys` de Process Explorer (Sysinternals) est :

- **Signé par Microsoft** (certificat Microsoft Windows Hardware Compatibility Publisher)
- Conçu pour obtenir des handles `PROCESS_ALL_ACCESS` et terminer des processus depuis le kernel
- Utilisé par l'outil d'attaque **AuKill/AvNeutralizer** (attribué à FIN7) pour tuer des EDR en production
- Dérivé de l'outil open-source **Backstab** (2021)

Historique des versions :

Version	Statut
≤ 16.x (ex: 16.32)	Exploitable — donne `PROCESS_ALL_ACCESS` sur tout processus, y compris PPL. Bloqué par la WDAC blocklist depuis 2023.
17.0+	Durci — vérifie `PsIsProtectedProcess`, restreint l'accès aux processus PPL
17.02	Encore utilisé par AvNeutralizer (FIN7) contre les processus non-PPL. **Non bloqué** par la WDAC.

⚠ Les attaquants embarquent la version **16.32** dans leur malware. Même bloquée par la WDAC, cette blocklist n'est pas activée par défaut sur tous les systèmes.

Vous allez analyser ce driver et reproduire la technique dans votre VM.

Partie A — Extraction et analyse du driver (30min)

Étape 1 : Obtenir le driver

1. Télécharger **Process Explorer** depuis Sysinternals
2. Lancer `procexp64.exe` une première fois **en administrateur** — cela extrait le driver dans :
   - `C:\Windows\System32\drivers\PROCEXP152.sys`
3. Copier le `.sys` dans votre répertoire de travail pour l'analyser

Étape 2 : Analyser le binaire PE

```cmd
dumpbin /headers PROCEXP152.sys
```

Q8 : Remplissez le tableau suivant dans votre rapport :

Champ	Valeur
Subsystem	
Signé par	
Taille du fichier	
Sections PE (.text, .rdata, PAGE, INIT)	

Vérifiez la signature numérique :

```powershell
```

```
Get-AuthenticodeSignature C:\Windows\System32\drivers\PROCEXP152.sys
```

Q9 : Qui est le signataire (Subject) du certificat ? Le certificat est-il encore valide ? Pourquoi Microsoft ne peut-il pas simplement révoquer ce certificat ou ajouter ce driver à la blocklist ?

Étape 3 : Analyser dans WinDBG

Avec Process Explorer lancé (et donc le driver chargé) :

```
code

kd> !drvobj \Driver\PROCEXP152 2
kd> !devobj \Device\PROCEXP152
```

Q10 : Listez les `MajorFunction` enregistrées par le driver. Lesquelles suggèrent qu'il accepte des commandes depuis le userland ? Quel est le nom du device créé dans le namespace ?

Partie B — Écrire un « process killer » avec procexp.sys (45min)

Objectif

Écrire un programme C `procexp_kill.c` qui utilise le driver PROCEXP152 pour terminer un processus **sans avoir les droits de le tuer directement**. Ce programme simule la chaîne d'attaque de l'outil **AuKill**.

Pourquoi le driver est nécessaire

Normalement, pour tuer un processus, il faut obtenir un handle avec le droit `PROCESS_TERMINATE` via `OpenProcess`. Mais si le processus cible est protégé (par `ObRegisterCallbacks`, PPL, ou autre), cet appel échoue avec `ERROR_ACCESS_DENIED`.

Le driver PROCEXP152 **contourne cette restriction** car il opère en **kernel mode** : il appelle `PsLookupProcessByProcessId` puis `ObOpenObjectByPointer` directement dans le noyau, ce qui **bypass les ObCallbacks** userland.

Chaîne d'attaque

```
code

Userland (sans droits)          PROCEXP152.sys (kernel)
──────────────────────          ──────────────────────────
1. OpenProcess(PROCESS_         ← Échec: Access Denied
   TERMINATE) → FAIL               (ObCallbacks bloque)

2. DeviceIoControl(             → Le driver reçoit le PID
   IOCTL 0x8335003C,            → PsLookupProcessByProcessId
   PID)                         → ObOpenObjectByPointer
                                → Retourne PROCESS_ALL_ACCESS
                                ← Handle kernel renvoyé
```

```
3. TerminateProcess(handle)    → Succès !
```

IOCTLs documentés

Les IOCTLs de PROCEXP152 sont documentés dans la recherche de sécurité (Sophos, SentinelOne, Backstab) :

IOCTL	Code	Rôle
`IOCTL_OPEN_PROTECTED_PROCESS_HANDLE`	`0x8335003C`	Obtenir un handle `PROCESS_ALL_ACCESS` sur un processus par son PID (bypass ObCallbacks)
`IOCTL_CLOSE_HANDLE`	`0x83350004`	Fermer un handle dans le contexte d'un processus cible

Ce que vous devez implémenter

c

```c
#include <windows.h>
#include <stdio.h>

#define PROCEXP_DEVICE L"\\\\.\\PROCEXP152"

// IOCTLs documentés (Sophos/SentinelOne/Backstab)
#define IOCTL_OPEN_PROTECTED_PROCESS_HANDLE  0x8335003C
#define IOCTL_CLOSE_HANDLE                   0x83350004

int wmain(int argc, wchar_t *argv[])
{
    if (argc < 2) {
        wprintf(L"Usage: %s <PID>\n", argv[0]);
        wprintf(L"Kills a process using PROCEXP152 driver (BYOLD)\n");
        return 1;
    }

    DWORD targetPid = _wtoi(argv[1]);

    // --- Démonstration : prouver qu'on n'a PAS les droits ---
    // TODO: Tenter OpenProcess(PROCESS_TERMINATE, FALSE, targetPid)
    //   → Si ça réussit : le processus n'est pas protégé, on n'a
    //     pas besoin du driver (fermer le handle, continuer quand même)
    //   → Si ça échoue (ERROR_ACCESS_DENIED) : c'est le cas intéressant

    // TODO: Vérifier l'existence du processus avec le droit minimal :
    //   OpenProcess(PROCESS_QUERY_LIMITED_INFORMATION, FALSE, targetPid)
    //   → PROCESS_QUERY_LIMITED_INFORMATION (0x1000) est le droit
    //     minimal pour vérifier qu'un processus existe.
    //     Ce droit ne permet PAS de le terminer.

    // --- Attaque via le driver ---
    // TODO: Ouvrir le device PROCEXP152 avec CreateFileW

    // TODO: Envoyer IOCTL_OPEN_PROTECTED_PROCESS_HANDLE avec le PID
    //   en entrée. Le driver contourne les protections userland et
```

```
    //   retourne un handle PROCESS_ALL_ACCESS.

    // TODO: Appeler TerminateProcess(handle, 1) avec le handle
    //   obtenu du driver → doit réussir même si OpenProcess avait échoué

    // TODO: Logger et comparer les résultats :
    //   - OpenProcess(TERMINATE) : échoué/réussi ?
    //   - IOCTL → TerminateProcess : échoué/réussi ?

    return 0;
}
```

⚠ **Note** : les structures exactes des buffers d'entrée/sortie peuvent varier selon la version de Process Explorer. Si les IOCTLs ci-dessus ne fonctionnent pas directement, utilisez le fuzzing (Partie 4) et WinDBG pour observer la taille des buffers attendue. La démarche de reverse engineering fait partie de l'exercice.

Comportement selon la version

Version	`OpenProcess(TERMINATE)`	IOCTL `0x8335003C`	`TerminateProcess(handle)`
v16.x	Dépend des protections	`PROCESS_ALL_ACCESS` (tout processus)	**Réussit toujours**
v17.x (cible non-PPL)	Dépend des protections	`PROCESS_ALL_ACCESS`	**Réussit**
v17.x (cible PPL)	Échoue	`PROCESS_QUERY_LIMITED_INFORMATION` seulement	**Échoue** (droits insuffisants)

Vérification

Scénario 1 : processus non protégé

1. Lancer `notepad.exe` et noter son PID
2. Vérifier que Process Explorer est lancé (driver chargé)
3. Exécuter `procexp_kill.exe <PID_notepad>`
4. Observer : `OpenProcess(TERMINATE)` réussit (notepad n'est pas protégé) ET l'IOCTL réussit aussi → les deux chemins fonctionnent

Scénario 2 : processus protégé (anticipation du TP 03)

Ce scénario sera pleinement testable après le TP 03 quand vous aurez votre driver `protector.sys` avec `ObRegisterCallbacks`. Pour l'instant, documentez le scénario théorique :

1. Un processus est protégé par `ObRegisterCallbacks` (masque `PROCESS_TERMINATE` retiré)
2. `OpenProcess(PROCESS_TERMINATE)` → **échoue** (Access Denied)
3. `OpenProcess(PROCESS_QUERY_LIMITED_INFORMATION)` → **réussit** (droit minimal, non filtré)
4. IOCTL `0x8335003C` → le driver **contourne** ObCallbacks → retourne `PROCESS_ALL_ACCESS`
5. `TerminateProcess(handle)` → **réussit**

Q11 : Exécutez `procexp_kill.exe` sur notepad. Faites une capture d'écran montrant :

- Le résultat de `OpenProcess(PROCESS_TERMINATE)` (réussi ou échoué)
- Le résultat de `OpenProcess(PROCESS_QUERY_LIMITED_INFORMATION)` (réussi)

- Le handle retourné par l'IOCTL
- La terminaison via `TerminateProcess`

Q12 (réflexion) : Expliquez dans votre rapport pourquoi `PROCESS_QUERY_LIMITED_INFORMATION` (0x1000) n'est pas filtré par `ObRegisterCallbacks` alors que `PROCESS_TERMINATE` l'est. Quel est le risque si on bloquait aussi ce droit minimal ?

Partie C — Fuzzing de procexp.sys (15min)

Utilisez votre fuzzer de la Partie 4 (ou un fuzzer simple) pour découvrir les IOCTL acceptés par `PROCEXP152` :

```cmd
ioctl_fuzzer.exe \\.\PROCEXP152 500
```

Q12 : Parmi les IOCTL testés, combien retournent `STATUS_SUCCESS` ? Listez les function codes qui répondent positivement. En déduisez-vous d'autres fonctionnalités exposées par le driver ?

Partie D — Analyse de la menace et des défenses (15min)

Q13 : Répondez aux questions suivantes dans votre rapport :

1. **Pourquoi ce driver est-il dangereux ?** Comparez avec un BYOVD classique (RTCore64.sys). Quels avantages a l'attaquant avec un BYOLD ?
2. **Défenses possibles** : recherchez quelles protections existent :
   - La **Microsoft Vulnerable Driver Blocklist** bloque-t-elle PROCEXP152.sys ? Pourquoi ?
   - Vérifiez sur votre VM : `Get-CimInstance -ClassName Win32_DeviceGuard -Namespace root\Microsoft\Windows\DeviceGuard`
   - Comment une entreprise pourrait-elle se protéger contre cette attaque ? (Hint : WDAC, règles custom)
3. **Lien avec le TP 03** : le driver Process Protector (ObRegisterCallbacks) que vous développerez au TP 03 peut-il empêcher procexp.sys de tuer un processus protégé ? Formulez une hypothèse.

Partie 4 — Fuzzing d'IOCTL en C userland (1h30)

Contexte

Les vulnérabilités des drivers Windows sont souvent découvertes par **fuzzing des IOCTL** : on envoie des `DeviceIoControl` avec des codes IOCTL, tailles et données aléatoires pour provoquer des crashes ou des comportements inattendus.

C'est un exercice de **programmation C userland** — vous n'avez pas besoin de modifier le kernel. Vous allez écrire un fuzzer qui cible le device exposé par un driver chargé.

Objectifs

Écrire un programme `ioctl_fuzzer.c` en C qui :

1. Ouvre un handle vers un device driver via `CreateFileW`
2. Génère des IOCTL codes aléatoires (en respectant la structure `CTL_CODE`)
3. Envoie des `DeviceIoControl` avec des buffers de tailles et contenus variables
4. Logue les résultats (succès, erreur, code retour) dans la console

Rappel : structure d'un CTL_CODE

```c
#define CTL_CODE(DeviceType, Function, Method, Access) \
    (((DeviceType) << 16) | ((Access) << 14) | ((Function) << 2) | (Method))
```

Un IOCTL code est un DWORD composé de 4 champs. Pour fuzzer, on peut :

- Fixer `DeviceType` (ex: `FILE_DEVICE_UNKNOWN` = 0x22)
- Itérer sur `Function` (0x000 à 0xFFF)
- Tester les 4 méthodes (0 à 3)
- Fixer `Access` (ex: `FILE_ANY_ACCESS` = 0)

Ce que vous devez implémenter

```c
#include <windows.h>
#include <winioctl.h>
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// TODO: Implémenter generate_ioctl_code(DWORD function, DWORD method)
// qui construit un CTL_CODE avec DeviceType=FILE_DEVICE_UNKNOWN,
// le function code et la method passés en paramètre, et FILE_ANY_ACCESS

// TODO: Implémenter generate_random_buffer(PUCHAR buffer, DWORD size)
// qui remplit un buffer avec des données aléatoires (rand())

int main(int argc, char *argv[])
{
    if (argc < 2) {
```

```
        printf("Usage: %s <device_path> [iterations]\n", argv[0]);
        printf("Example: %s \\\\.\\HelloDriver 1000\n", argv[0]);
        return 1;
    }

    char *devicePath = argv[1];
    int iterations = (argc > 2) ? atoi(argv[2]) : 100;

    srand((unsigned int)time(NULL));

    // TODO: Convertir devicePath en wide string et ouvrir le device
    //       avec CreateFileW (GENERIC_READ | GENERIC_WRITE)

    // TODO: Boucle de fuzzing sur 'iterations' itérations :
    //   1. Générer un function code aléatoire (0 à 0xFFF)
    //   2. Générer une method aléatoire (0 à 3)
    //   3. Construire l'IOCTL code via generate_ioctl_code()
    //   4. Générer un buffer d'entrée de taille aléatoire (0 à 4096)
    //   5. Remplir le buffer avec des données aléatoires
    //   6. Appeler DeviceIoControl avec un timeout (ou dans un thread)
    //   7. Logger : numéro d'itération, IOCTL code (hex),
    //      taille du buffer, résultat (succès/erreur), GetLastError()
    //   8. Si le DeviceIoControl réussit avec des données en sortie,
    //      afficher les premiers octets du buffer de sortie

    // TODO: Fermer le handle et afficher les statistiques
    //       (nombre de succès, nombre d'erreurs, codes d'erreur uniques)

    return 0;
}
```

Compiler avec :

```
code

cl ioctl_fuzzer.c /Fe:ioctl_fuzzer.exe
```

Vérification

Test sur votre Hello Driver

Votre `HelloDriver` du TP ne crée pas de device (pas de `IoCreateDevice`). Que se passe-t-il si on tente d'ouvrir `\\.\HelloDriver` ?

Q14 : Testez `ioctl_fuzzer.exe \\.\HelloDriver 10`. Quel est le résultat ? Quel code d'erreur retourne `CreateFileW` ? Expliquez pourquoi.

Test sur procexp.sys

C'est ici que la Partie 3 et la Partie 4 convergent. Utilisez votre fuzzer sur le driver Process Explorer :

```
code
```

```
ioctl_fuzzer.exe \\.\PROCEXP152 500
```

Q15 : Comparez les résultats du fuzzing de `PROCEXP152` avec ceux de `PhysicalDrive0`. Quel driver expose plus d'IOCTL fonctionnels ? Reportez les function codes qui retournent `STATUS_SUCCESS` sur PROCEXP152.

Observation dans WinDBG

Pendant l'exécution du fuzzer sur PROCEXP152, posez un breakpoint dans WinDBG :

```
code

kd> bp PROCEXP152!DispatchDeviceControl
```

Si le symbole n'est pas résolu (driver non public) :

```
code

kd> !drvobj \Driver\PROCEXP152 2
kd> bp <adresse_dispatch_from_MajorFunction[14]>
```

Q16 : Combien de fois le breakpoint est atteint pour 100 itérations du fuzzer ? Observez le contenu de l'IRP à chaque hit. Faites une capture d'écran.

Pour aller plus loin (bonus)

Fuzzer intelligent

Améliorez votre fuzzer avec :

- **Logging dans un fichier CSV** : horodatage, IOCTL, taille, résultat, durée
- **Mode cartographie** : pour chaque IOCTL qui retourne `STATUS_SUCCESS`, tester avec des buffers de taille croissante (0, 1, 2, 4, 8, ..., 65536) pour comprendre la taille attendue
- **Corrélation avec procexp_kill** : vérifier que l'IOCTL de kill découvert par fuzzing correspond à celui utilisé dans votre programme de la Partie 3

Q17 (bonus) : Implémentez le mode cartographie et montrez un extrait du log CSV. Combien d'IOCTL distincts accepte PROCEXP152 ?

Barème indicatif

Élément	Points
Partie 1 — Setup fonctionnel + driver Hello (Q1–Q3)	3

Partie 2 — Commandes WinDBG (Q4–Q7)	4
Partie 3 — BYOLD procexp.sys (Q8–Q13)	7
Partie 4 — Fuzzer fonctionnel (Q14–Q16)	4
Bonus — Fuzzer intelligent (Q17)	2
Total	**20**

Annexe — Compilation sans Visual Studio

Voir le document **« Annexe — Compilation de drivers en ligne de commande »** (`annexe_compilation_cmdline.pdf`) pour compiler vos drivers avec `nmake` et `cl.exe` sans ouvrir Visual Studio.

Trois méthodes y sont documentées : **EWDK** (ISO autonome), **VS Build Tools + MSBuild**, et **nmake + Makefile manuel** avec tous les flags expliqués.

Troubleshooting

Problème	Solution
`sc start` retourne « Access denied »	Lancer le terminal en **Administrateur**
`sc start` retourne erreur de signature	Vérifier que `bcdedit /set testsigning on` a été appliqué + reboot
WinDBG ne se connecte pas	Vérifier IP/port/key, désactiver le firewall Windows sur la VM
DbgView n'affiche rien	Activer « Capture Kernel » dans le menu Capture de DbgView
`lm` ne montre pas le driver	Le driver n'est pas encore chargé — faire `sc start` d'abord
Symboles non résolus dans WinDBG	Configurer le symbol path : `srv*C:\Symbols*https://msdl.microsoft.com/download/symbols`
`CreateFileW` retourne ERROR_FILE_NOT_FOUND	Le device n'existe pas — le driver ne crée pas de device ou le nom est incorrect
Le fuzzer freeze	Le `DeviceIoControl` bloque — ajouter un timeout via thread ou `CancelIoEx`
BSOD pendant le fuzzing	C'est le but ! Le fuzzer a trouvé un crash. Noter l'IOCTL code et le bugcheck code.